

1 Splines/Spline.h

- Line 23: `#include "eigen3/Eigen/Eigen"`
- Line 111: `void initialize_common_part(Eigen::MatrixXd &A, Eigen::VectorXd &rhs,`
`&rhs,`
- Line 115: `void assign_polys_with_M(const Eigen::VectorXd &M, const std::vector<double>`
`&values);`
- Line 117: `void complete(bool left, Eigen::MatrixXd &A, Eigen::VectorXd &rhs,`
- Line 120: `void specified_2nd(bool left, Eigen::MatrixXd &A, Eigen::VectorXd`
`&rhs, double);`
- Line 122: `void natural(bool left, Eigen::MatrixXd &A, Eigen::VectorXd &rhs);`
- Line 124: `void not_a_knot(bool left, Eigen::MatrixXd &A, Eigen::VectorXd`
`&rhs);`
- Line 126: `void periodic(Eigen::MatrixXd &A, Eigen::VectorXd &rhs,`
- Line 196: `void init_matrix(Eigen::MatrixXd &A, Eigen::VectorXd &rhs, const`
`std::vector<double> &values);`

2 Splines/Spline.cpp

- Line 74: `void Spline<3, SplineType::ppForm>::initialize_common_part(Eigen::MatrixXd`
`&A, Eigen::VectorXd &rhs,`
- Line 87: `void Spline<3, SplineType::ppForm>::assign_polys_with_M(const Eigen::VectorXd`
`&M,`
- Line 110: `Eigen::MatrixXd A = Eigen::MatrixXd::Zero(N, N);`
- Line 113: `Eigen::VectorXd rhs = Eigen::VectorXd::Zero(N);`
- Line 165: `Eigen::VectorXd M = A.lu().solve(rhs);`
- Line 169: `void Spline<3, SplineType::ppForm>::complete(bool left, Eigen::MatrixXd`
`&A, Eigen::VectorXd &rhs,`
- Line 184: `void Spline<3, SplineType::ppForm>::specified_2nd(bool left, Eigen::MatrixXd`
`&A, Eigen::VectorXd &rhs, double d) {`

- Line 196: `void Spline<3, SplineType::ppForm>::natural(bool left, Eigen::MatrixXd &A, Eigen::VectorXd &rhs) {`
- Line 200: `void Spline<3, SplineType::ppForm>::not_a_knot(bool left, Eigen::MatrixXd &A, Eigen::VectorXd &rhs) {`
- Line 216: `void Spline<3, SplineType::ppForm>::periodic(Eigen::MatrixXd &A, Eigen::VectorXd &rhs,`
- Line 461: `void Spline<3, SplineType::BSpline>::init_matrix(Eigen::MatrixXd &A,`
- Line 462: `Eigen::VectorXd &rhs,`
- Line 465: `A = Eigen::MatrixXd::Zero(N + 2, N + 2);`
- Line 466: `rhs = Eigen::VectorXd::Zero(N + 2);`
- Line 484: `Eigen::MatrixXd A;`
- Line 485: `Eigen::VectorXd rhs;`
- Line 501: `Eigen::VectorXd res = A.lu().solve(rhs);`
- Line 511: `Eigen::MatrixXd A;`
- Line 512: `Eigen::VectorXd rhs;`
- Line 528: `Eigen::VectorXd res = A.lu().solve(rhs);`
- Line 546: `Eigen::MatrixXd A;`
- Line 547: `Eigen::VectorXd rhs;`
- Line 573: `Eigen::VectorXd res = A.lu().solve(rhs);`

3 Splines/SpapiSolver.h

- Line 7: `#include<eigen3/Eigen/Sparse>`
- Line 11: `using namespace Eigen;`
- Line 20: `SparseMatrix<double> M;`
- Line 88: `M = SparseMatrix<double>(m, m);`

- Line 90: `std::vector<Triplet<double>> tripletlist;`
- Line 94: `tripletlist.push_back(Triplet<double>(i, j, base[j].d(degree[i], x[i])));`
- Line 98: `M.setFromTriplets(tripletlist.begin(), tripletlist.end());`
- Line 99: `M.makeCompressed();`
- Line 100: `SparseQR<SparseMatrix<double>, NaturalOrdering<int>> solver;`
- Line 106: `VectorXd rhs(m);`
- Line 110: `VectorXd result = solver.solve(rhs);`